1. Introduzione

La presente relazione descrive il progetto realizzato per l’esame di Reti e Calcolatori.

Il progetto oggetto della relazione consiste nell’utilizzare il protocollo HTTP per rendere disponibili ai client, mediante un insieme di web API (cioè un insieme di url) le funzionalità di un sistema di compravendita di oggetti usati.

L'obiettivo preposto dal progetto riguarda l'implementazione, oltre alle web API del sistema, anche delle funzionalità di un server che le renda fruibili in rete rispondendo alle richieste sulla porta 80 e comunicando con i client attraverso messaggi HTTP. La comunicazione sarà basata appoggiandosi al solo livello di trasporto mediante l'utilizzo delle socket di tipo TCP.

1.1. Funzionalità del sistema

Le web API implementate sono relative a un sistema per la compravendita di oggetti indesiderati regalati durante una festività come quella Natalizia, consentendo ai venditori di rendere visibili i propri oggetti solamente alle persone che non fanno parte della propria cerchia di amicizia, al fine di ridurre il più possibile il rischio che gli amici scoprano che i propri regali non siano stati apprezzati.

1.1.1. Rete di amicizie degli utenti

Al fine di rendere il sistema capace di stabilire se tra due utenti c’è o meno un legame di amicizia, è stata implementata una rete di amicizie che collega gli utenti tra i quali esiste una amicizia, insieme alle funzionalità atte a consultarla.

1.1.2. Web API

Nei successivi capitoli vengono descritte le web API del sistema che sono state implementate. L’utilizzo di ciascuna delle web API implementate è limitato ai soli utenti iscritti, al fine di garantire che le operazioni degli utenti vengano nascoste ai relativi amici. Per questo motivo, ogni web API richiede, in aggiunta agli eventuali parametri di input necessari, che venga fornito l’username dell’utente che ne richiede l’utilizzo al fine di riconoscerlo.

1.1.2.1. Aggiunta di un prodotto in vetrina

Consente a un venditore di aggiungere alla propria vetrina un nuovo prodotto.

Informazioni	
URL	/AggiungiProdotto
Parametri richiesti:	
username	Username dell’utente che esegue l’operazione
nome	Nome col quale il prodotto viene aggiunto
prezzo	Prezzo col quale il prodotto viene aggiunto
Casi di errore:	
Uno o più parametri mancanti	
Prezzo non valido	
Utente con username fornito non esistente	

1.1.2.2. Acquisto di un prodotto in vetrina

Consente a un acquirente di comprare un prodotto in vetrina.

Il sistema, prima di processare l'acquisto, verifica che non esista un legame di amicizia tra l'acquirente e il proprietario del prodotto richiesto. In caso contrario, impedisce l'acquisto fornendo un errore.

Informazioni	
URL	/CompraProdotto
Parametri richiesti:	
username	Username dell'utente che esegue l'operazione
idProdotto	Id del prodotto da acquistare
Casi di errore:	
Uno o più parametri mancanti	
Utente con username fornito non esistente	
Prodotto con idProdotto fornito non esistente o appartenente a un amico dell'utente	
Prodotto con idProdotto fornito già venduto	
Prodotto con idProdotto fornito appartiene già all'utente	

1.1.2.3. Reperimento dei prodotti in vetrina

Consente all'utente di reperire l'elenco di tutti i prodotti in vetrina. L'elenco fornito dalla web API conterrà i soli prodotti messi in vetrina dagli utenti che non sono legati dalla relazione di amicizia con l'utente che esegue l'operazione.

Informazioni	
URL	/GetProdottiDisponibili
Parametri richiesti:	
username	Username dell'utente che esegue l'operazione
Casi di errore:	
Parametro username mancante	
Utente con username fornito non esistente	

1.1.2.4. Reperimento dei propri prodotti venduti

Fornisce all'utente l'elenco di tutti i prodotti che ha venduto.

Informazioni	
URL	/GetProdottiVenduti
Parametri richiesti:	
username	Username dell'utente che esegue l'operazione
Casi di errore:	
Parametro username mancante	

Utente con username fornito non esistente

1.2. Funzionalità del server

Nei seguenti capitoli vengono riportate le funzionalità relative al server che sono state implementate.

1.2.1. Accettare richieste di connessione

Il server deve mettersi in ascolto su una specifica porta al fine di accettare le richieste di connessione in arrivo dai client e aprire un canale di comunicazione con i client stessi.

1.2.2. Ricevere richieste HTTP

Il server deve leggere e interpretare le richieste di web API ricevute dal client in formato HTTP e processarle attivando le relative funzionalità del sistema.

La richiesta deve essere strutturata nel modo fornito di seguito.

```
GET <URL Risorsa con eventuali parametri> HTTP/1.0
<riga di header>
…
<riga di header>
<riga vuota>
```

1.2.3. Inviare risposte HTTP

Il server deve creare e inviare risposte HTTP a seguito del processamento delle richieste ricevute, e chiudere successivamente le relative connessioni.

Una risposta HTTP creata dal server ha il seguente formato:

```
HTTP/1.0 <STATUS_CODE> <STATUS_MESSAGE>
Date: <data e ora>
Content-Type: application/json
Content-Length: <lunghezza del corpo>
<riga vuota>
<body>
```

Ogni risposta generata dal server è costituita dalle sole seguenti informazioni di header:

- Date: il suo valore è settato alla data e ora di creazione della risposta;
- Content-Type: informazione che rappresenta il tipo del del corpo della risposta, e il suo valore è application/json, in quanto il corpo della risposta, qualora esista, è in formato JSON;
- Content-Length: informazione che rappresenta la lunghezza del corpo della risposta. Se non presente, il suo valore è 0;

1.2.4. Chiudere la connessione con il client

Il server chiude la connessione col client a termine dello scambio di messaggi HTTP col client stesso.

1.2.5. Processamento di richieste in modalità multithread

Il server processa in parallelo le connessioni con i client, mediante l’ausilio di un pool di un numero fissato di thread.

2. Implementazione

2.1. Architettura

La figura 2.1.1. Mostra l'architettura del sistema complessivo costituita dai componenti che gestiscono la comunicazione con i client e quelli che implementano le funzionalità associate delle web API. L'architettura, inoltre, descrive attraverso l'uso delle frecce le relazioni di dipendenza funzionale che sussistono tra i componenti stessi. I componenti sono stati realizzati mediante le classi Java e sono stati raggruppati in diversi package per risaltare il loro ruolo nella logica complessiva del sistema. L'organizzazione prevista viene illustrata nel dettaglio nei capitoli successivi.

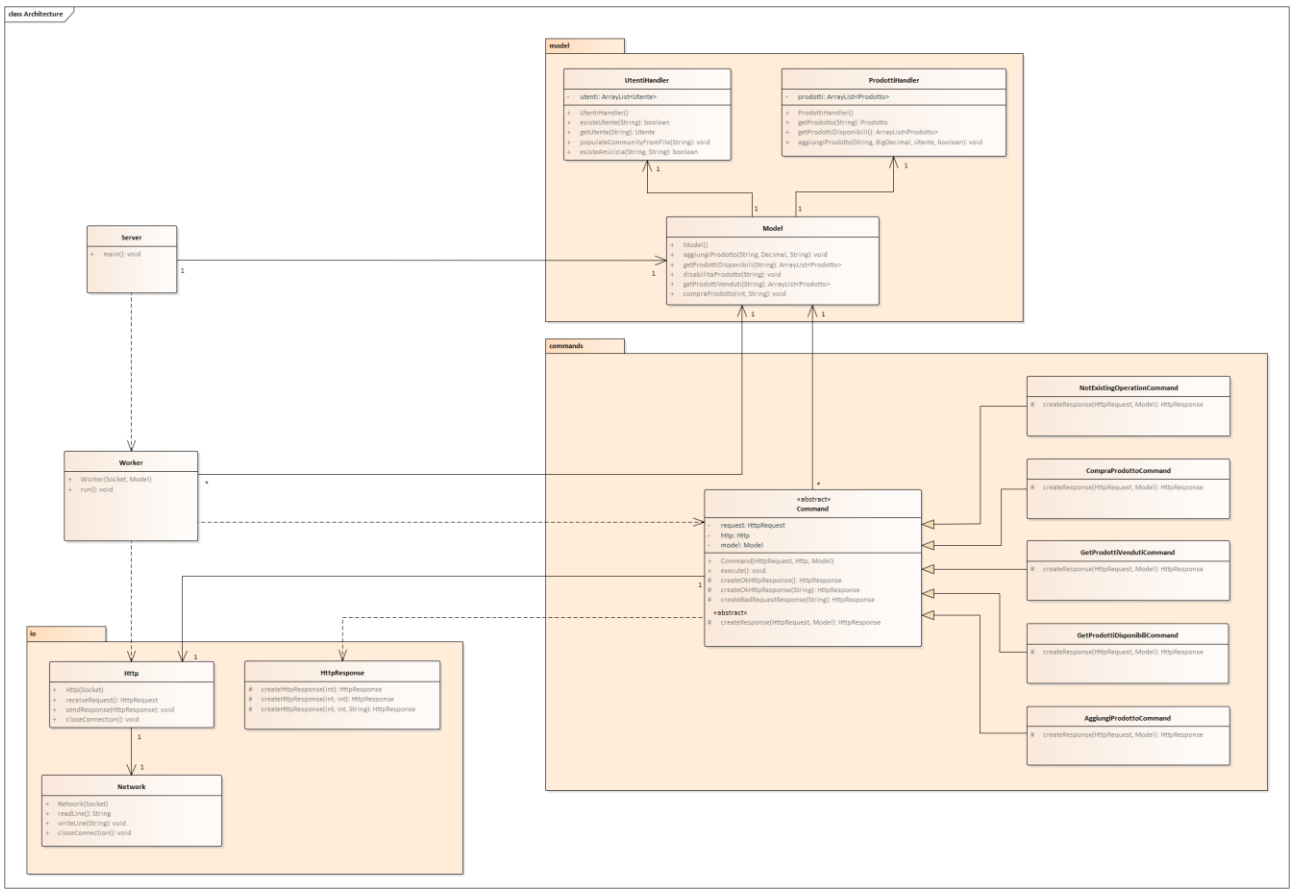

Figura 2.1.1.: architettura del sistema

2.1.1. Package io

I componenti di questo package si occupano dell'interscambio dei messaggi HTTP sul canale di comunicazione tra client e server.
Di questo package fanno parte i seguenti componenti:
- Network: astrae l'invio e la ricezione dei singoli byte a livello di trasporto interagendo direttamente con la socket TCP del server. Fornisce così al livello applicativo le funzionalità di invio e ricezione di intere righe di testo al client mediante i metodi writeLine() e readLine(), oltre alla funzionalità di chiusura della connessione con il metodo closeConnection().
- Http: astrae il livello applicativo inviando e ricevendo messaggi HTTP tra il server e il client mediante i metodi sendResponse() e receiveRequest() e chiudendo la connessione tra il server e il client col metodo closeConnection(). Le sue funzionalità sono implementate con l'ausilio di quelle del Network; i messaggi HTTP vengono infatti inviati e ricevuti riga per riga, e la connessione viene chiusa usufruendo della funzionalità correlata che agisce a livello di trasporto;

- HttpResponse: con le varianti del suo metodo createHttpResponse(), fornisce le utility necessarie a creare una istanza della medesima classe che rappresenta una risposta HTTP da inviare al client.

2.1.2. Package model

I componenti del package model implementano le funzionalità messe a disposizione dalle web API, quindi agiscono sul modello dei dati dell'applicazione rendendo effettive le operazioni attuate dagli utenti durante l'utilizzo dell'applicazione stessa. come il reperimento e l'inserimento dei prodotti in vetrina e l'acquisto dei prodotti.

2.1.3. Package commands

I componenti del package commands sono responsabili del processamento delle richieste HTTP che vengono ricevute dal server. Fanno parte di questo package la classe astratta Command, rappresentante in modo astratto il comando da eseguire associato a una generica web API che l'utente chiama, e tutte le classi concrete che estendono Command, ciascuna di quest'ultime atta ad implementare il comando concreto associato a una specifica web API dell'applicazione. Command è il componente più importante del package perchè fornisce alle classi concrete il metodo execute() per eseguire i comandi richiesti dagli utenti e caratterizza, seppur in maniera astratta, il comportamento assunto da ogni comando nel processamento della richiesta, interagendo da un lato con i componenti del modello per ottenere il risultato dell'operazione e dall'altro con i componenti di input/output per incapsulare il risultato in una risposta HTTP e inviarla successivamente ai client.

2.1.4. Componenti Server e Worker

Server è il componente fondamentale del sistema in quanto fa partire l'esecuzione del server mettendolo in ascolto sulla porta 80 per accettare le richieste di connessione in arrivo dai client e delegarne il processamento in parallelo con l'ausilio un pool di thread di dimensione fissa. Inoltre, esegue le operazioni preliminari all'avvio del server, come la creazione del componente Model e del pool di thread.
Worker è il componente al quale il server delega la ricezione, l'interpretazione e il soddisfacimento della richiesta HTTP.

2.2. Realizzazione della rete di amicizie

La rete di amicizie degli utenti è stata implementata mediante un grafo in cui ogni utente è rappresentato da uno specifico nodo e l'esistenza di amicizia tra due utenti è rappresentata da un arco che congiunge i rispettivi nodi. Tale grafo è non orientato, in quanto l'amicizia tra due utenti è simmetrica, cioè dati due utenti a e b, a è amico di b se e solo se b è amico di a.
Per tenere in memoria il grafo di amicizie è stato scelto l'utilizzo di una matrice di adiacenza: una matrice quadrata binaria avente sia come indici riga che indici colonna gli utenti e tale per cui ogni elemento (i, j) della matrice ha valore 1 se esiste una amicizia tra gli utenti i e j, 0 altrimenti.
Essendo il grafo non orientato, l'elemento (i, j) = 1 se e solo se l'elemento (j, i) = 1.

2.3. Funzionalità di rete

2.3.1. Connessione tra client e server

Il server, una volta messosi in ascolto di richieste di connessione in arrivo dai client su una determinata porta, è in grado di accettarle. Il diagramma in figura 2.2.1.1. descrive la sequenza di

operazioni eseguite dai componenti dal sistema per mettere il server in ascolto sulla porta 80 e accettare una richiesta di connessione.
Oltre al componente Server e al client che inoltra la richiesta di connessione, nel diagramma viene illustrata l'interazione con i componenti ServerSocket e Socket, facenti parte della libreria messa a disposizione da Java.

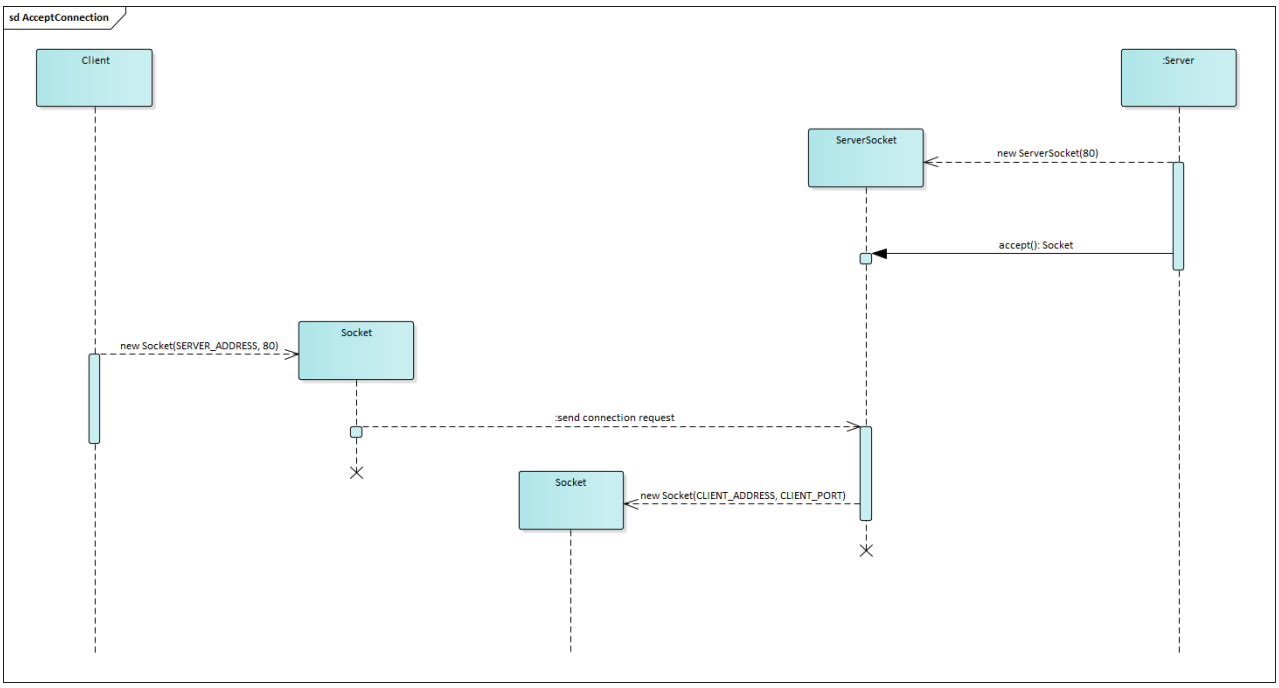

Figura 2.2.1.1: Sequence delle operazioni effettuate per accettare una richiesta di connessione

Il Server si mette in ascolto chiamando il metodo accept() della classe ServerSocket. Tale metodo è bloccante, in quanto sospende l'esecuzione del server fino a quando non arriva una richiesta di connessione effettuata dalla socket del client. Alla ricezione di una richiesta, il metodo accept() crea una nuova socket che il server potrà utilizzare per comunicare con la socket del client, al fine di ricevere da quest'ultimo una richiesta HTTP e di inviare la relativa risposta HTTP.

2.3.2. Scambio di messaggi HTTP tra client e server

La ricezione e il soddisfacimento della richiesta HTTP sono successivi alla creazione del canale di comunicazione tra la socket del client e quella del server descritta nel capitolo 2.2.1.
Il diagramma in figura 2.2.2.1 mostra, nell'ordine, la sequenza di operazioni che vengono attuate dai componenti del sistema dopo aver stabilito la connessione col client. Questa sequenza include l'utilizzo, oltre che del componente Socket, anche del componente ExecutorService facente parte della libreria Java e che rappresenta il pool di thread responsabile della esecuzione del Worker.

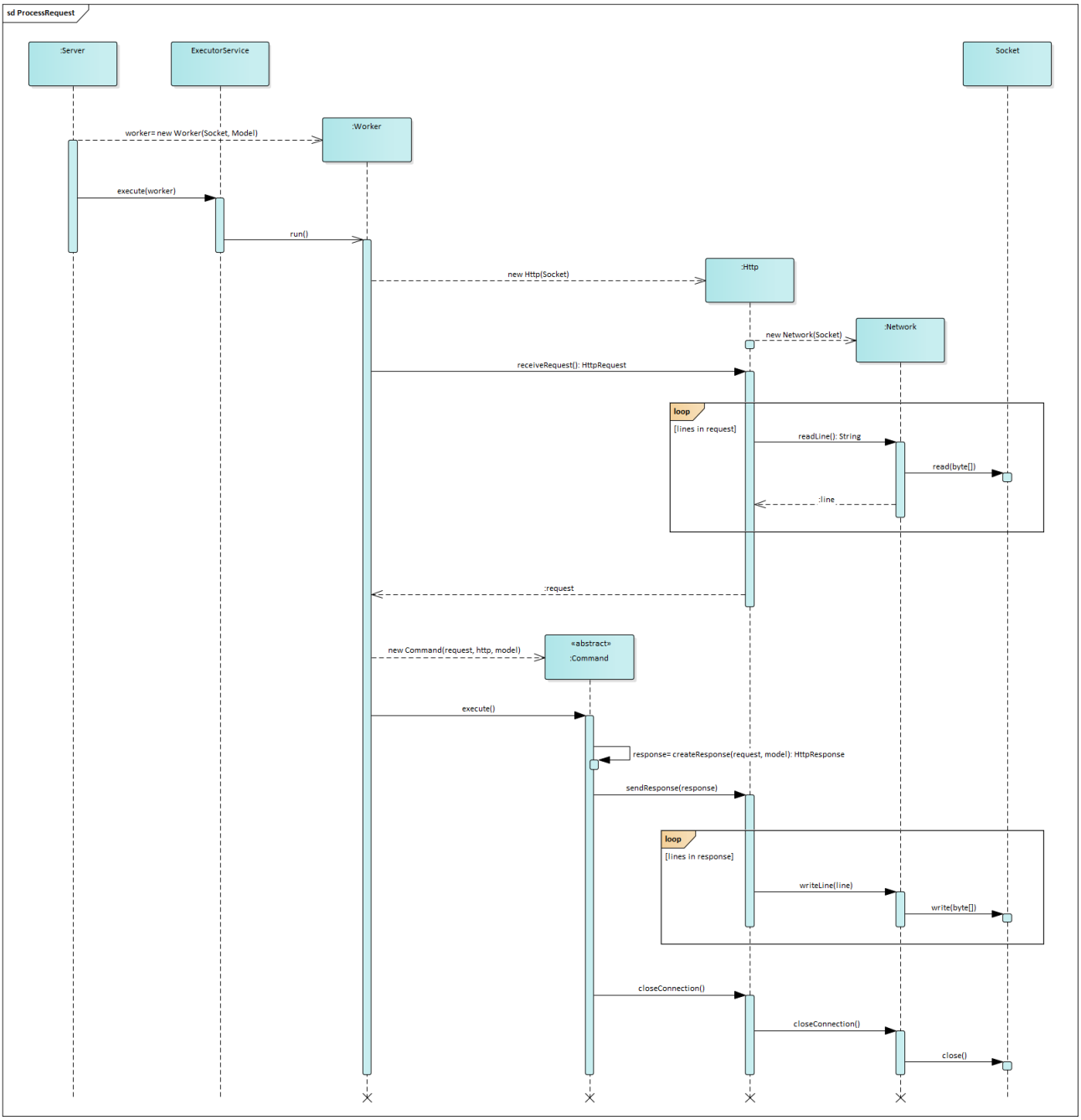

Figura 2.2.1.1: Sequence delle operazioni effettuate per processare la comunicazione con un client

Il Server crea dapprima un Worker fornendogli la propria socket e il Model sul quale attuare l'operazione che verrà richiesta dal client, e successivamente ne delega l'esecuzione al componente ExecutorService. Quando verrà eseguito, il Worker effettuerà, nell'ordine, le seguenti operazioni:

1. Creazione del componente Http fornendo la socket del server: il componente Http, nel momento della sua creazione, crea il componente Network al quale si affiderà per ricevere e inviare i messaggi HTTP riga per riga e per chiudere la connessione col client mediante la socket del server;
2. Ricezione della richiesta HTTP: viene effettuata chiamando il metodo receiveRequest() del componente Http. Tale metodo legge una per una le righe di testo che costituiscono la richiesta chiamando per ciascuna di esse il metodo readLine() del componente Network e successivamente crea un oggetto della classe HttpRequest contenente tutte le informazioni della richiesta ricevuta, fornendolo poi al Worker;

3. Creazione del componente Command associato alla web API dell'applicazione richiesta dal client, corredandolo della richiesta HTTP ricevuta dal client, del componente Http e del componente Model su cui attuare l'operazione associata alla web API richiesta. Nel diagramma, a fini di sintesi, viene astratta l'instanziazione della specifica classe che estende Command e che viene eseguita sulla base della specifica web API.
4. Esecuzione del componente Command creato: il Worker esegue il comando chiamando su quest'ultimo il metodo execute(). Tale metodo attua sul Model l'operazione richiesta, dopo aver prelevato dalla richiesta HTTP gli eventuali parametri necessari. Successivamente, crea un oggetto della classe HttpResponse che rappresenta la risposta HTTP contenente il risultato dell'operazione richiesta e la invia al client chiamando il metodo sendResponse() del componente Http di cui infine chiama il metodo closeConnection() per chiudere la connessione col client.

2.4. Semplificazioni introdotte nell'utilizzo dell'HTTP

Per la realizzazione delle funzionalità del server, sono state introdotte delle semplificazioni sull'utilizzo del protocollo HTTP, che vengono riportate di seguito.

2.4.1. Implementazione della versione HTTP/1.0

E' stata adottata questa versione del protocollo HTTP in quanto, a dispetto della mancanza delle funzionalità più avanzate offerte dalle versioni successive, quali quelle di sicurezza presenti nella versione HTTP/2.0, consente di focalizzare l'obiettivo del progetto sull'invio e la ricezione di messaggi mediante il livello di trasporto e l'interpretazione dei messaggi in arrivo.

2.4.2. Supporto delle sole richieste HTTP di tipo GET

La tipologia di richieste supportate dal server è ristretta alle sole richieste di tipo GET.

2.4.3. Supporto per un sottoinsieme limitato di codici di stato delle risposte

Nell'ambito della creazione di una risposta HTTP vengono supportati solo i seguenti codici di stato:

- 200 OK: Utilizzato per comunicare al client che la richiesta è stata processata con successo, senza alcun errore;

- 400 BAD REQUEST: Utilizzato per comunicare al client che la richiesta non è stata processata a causa di parametri non validi (e.g. richiesto di acquisto fornendo l'id di un prodotto non esistente);

- 404 NOT FOUND: Utilizzato per comunicare al client che la richiesta richiede una web API non esistente.

2.4.4. Utilizzo limitato delle informazioni di header

Le informazioni di header contenute nelle richieste HTTP vengono lette ma ignorate dal server. Inoltre, le risposte HTTP che il server crea contengono un insieme di informazioni di header limitato a quelle elencate nel capitolo 1.2.3.